Python Engineering - Class 03 Reference Guide

Bucles e iteracion

Guía de consulta de sintaxis y herramientas utilizadas en esta clase. Incluye qué hace cada elemento, cómo se escribe, qué recibe o produce y cuándo conviene utilizarlo.

Mapa rápido

Concepto	Sintaxis	Idea
for	for i in range(...):	Repite un bloque recorriendo los elementos de un iterable.
while	while condition:	Repite un bloque mientras una condición continúe siendo True.
range()	range(start, stop, step)	Genera una secuencia de enteros; el valor stop no se incluye.
Acumulador	total += value	Variable que conserva una suma, producto u otro resultado parcial durante las iteracione
Contador	i += 1	Variable utilizada para contar iteraciones o eventos.
Regla de parada	while h >= 0	Condición que determina cuándo debe finalizar un proceso iterativo.
División entera	//	División que devuelve el cociente entero cuando ese es el resultado que queremos cons
math.factorial()	factorial(n)	Implementación de referencia de Python utilizada para validar nuestro factorial iterativo.
math.exp()	exp(x)	Implementación de referencia de la exponencial utilizada para validar la aproximación de

1. for

Sintaxis: `for i in range(...):`

Repite un bloque recorriendo los elementos de un iterable.

```
for i in range(1, N + 1):
    total += i**2
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

2. while

Sintaxis: `while condition:`

Repite un bloque mientras una condición continúe siendo True.

```
while h >= 0:
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

3. range()

Sintaxis: `range(start, stop, step)`

Genera una secuencia de enteros; el valor stop no se incluye.

```
for i in range(1, 6):
    print(i)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

4. Acumulador

Sintaxis: `total += value`

Variable que conserva una suma, producto u otro resultado parcial durante las iteraciones.

```
total = 0
for x in values:
    total += x
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

5. Contador

Sintaxis: `i += 1`

Variable utilizada para contar iteraciones o eventos.

```
i = 0
i += 1
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

6. Regla de parada

Sintaxis: `while h >= 0`

Condición que determina cuándo debe finalizar un proceso iterativo.

```
while h >= 0:
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

7. División entera

Sintaxis: `//`

División que devuelve el cociente entero cuando ese es el resultado que queremos conservar.

```
exact_integer = numerator // denominator
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

8. math.factorial()

Sintaxis: `factorial(n)`

Implementación de referencia de Python utilizada para validar nuestro factorial iterativo.

```
reference = factorial(5)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

9. math.exp()

Sintaxis: `exp(x)`

Implementación de referencia de la exponencial utilizada para validar la aproximación de Taylor.

```
reference = exp(0.5)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

Errores comunes

- Confundir el nombre de una función con su llamada usando paréntesis.
- No revisar tipos, índices o shapes antes de una operación.
- Usar una herramienta porque funciona sin poder explicar qué recibe y qué devuelve.

Chuleta mental

```
for: for i in range(...):
while: while condition:
range(): range(start, stop, step)
Acumulador: total += value
Contador: i += 1
Regla de parada: while h >= 0
División entera: //
math.factorial(): factorial(n)
math.exp(): exp(x)
```